Specify Constructor of `std::nullopt_t`

Document #:	P3112R0
Date:	2024-02-14
Project:	Programming Language C++
Audience:	LEWG, LWG

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

1 Introduction

2 Why does this problem occur?

3 Proposed solution

4 Wording

5 References

§ 1 Introduction

Should the following code be well formed?

```
#include <optional>

struct Widget {
    class PassKey {
        PassKey() = default;
        friend class Widget;
    };
    Widget(PassKey) {}

    static std::optional<Widget> make() {
        std::optional<Widget> result{{{}}};
        return result;
    }
};
```

Because `{{}}` can be implicitly converted to `Widget`, one would expect this code to initialize `result` to hold a `Widget` object whose value is `Widget{{}}`. However, GCC 13.2 rejects this code, giving the following error message.

```
<source>:12:42: error: converting to 'std::nullopt_t' from initializer
        list     would     use     explicit     constructor     'constexpr
        std::nullopt_t::nullopt_t(_Construct)'
   12 |         std::optional<Widget> result{{{}}};
```

§ 2 Why does this problem occur?

An implementation detail has leaked from libstdc++: namely, that an implicit conversion sequence exists from `{{}}` to `std::nullopt_t` in libstdc++'s implementation of `std::nullopt_t`, because libstdc++ declares `std::nullopt_t` to have a constructor with a single parameter whose type is an implementation-private scoped enumeration (and a scoped enumeration can always be initialized from `{}`).

The attempt to use this implicit conversion sequence renders the program ill formed because the constructor that libstdc++ has declared for `std::nullopt_t` is explicit. This is not the behavior that most users would expect for the above code, which is to call the value constructor of `std::optional<Widget>`. The constructor of `std::optional<Widget>` that takes `std::nullopt_t` has won overload resolution because it is not a constructor template, while the value constructor is (§22.5.3.2 [optional.ctor][1]p23).

Clang and MSVC accept the above code. In libc++, the constructor of `std::nullopt_t` takes two arguments, not one. In both Clang and MSVC, the fact that `std::nullopt_t` has an explicit constructor prevents the formation of an implicit conversion sequence, but it is unclear whether it is correct for it to do so; see [CWG2525].

The standard has the following to say about how `std::nullopt_t` can be constructed (§22.5.4 [optional.nullopt]p2):

> Type `nullopt_t` shall not have a default constructor or an initializer-list constructor, and shall not be an aggregate.

These restrictions are not strong enough to prevent the formation of an implicit conversion sequence from a *braced-init-list* to `std::nullopt_t`, because list-initialization can call a non-initializer-list constructor. libstdc++'s implementation, in which `std::nullopt_t` has a constructor taking a single object of a tag type that is itself default-constructible, conforms to the standard. The following implementation would also be conforming, even though it makes it even easier for users to encounter the compilation error described above.

```
namespace std {
struct nullopt_t {
    nullopt_t(int) {}  // not explicit
};
}
```

§ 3 Proposed solution

We could choose to do nothing, and hope that the CWG will resolve CWG2525 in favor of Clang and MSVC, which would make GCC, with libstdc++'s current implementation of `std::nullopt_t`, accept the code shown in the introduction. On the other hand, if CWG2525 is resolved in GCC's favor, then the code will become ill formed with MSVC's current implementation of `std::nullopt_t`, as well as when Clang uses libstdc++.

Instead, I propose a simple solution that will completely specify how `std::nullopt_t` is constructed and, regardless of the disposition of CWG2525, guarantee that `std::nullopt_t` will not interfere when a user tries to construct `std::optional<T>` from a *braced-init-list*. We can declare a tag type from which `std::nullopt_t` is constructed, and make the actual constructor a template that accepts only that tag type. Such a constructor will never be a candidate for construction from a *braced-init-list*, because deduction can never succeed.

§ 4 Wording

The proposed wording is relative to [N4971].

In §22.5.4 [optional.nullopt], edit the code before p1 as follows.

```
struct nullopt_t{see below};
inline constexpr nullopt_t nullopt(unspecified);
```

```
struct nullopt_t {
  struct nullopt-construct-tag {};  // exposition only
    constexpr  explicit  nullopt_t(same_as<nullopt-construct-tag>
         auto) {}
};
inline constexpr nullopt_t nullopt(nullopt_t::nullopt-construct-
         tag());
```

Strike p2 in §22.5.4 [optional.nullopt]:

— ~~Type `nullopt_t` shall not have a default constructor or an initializer-list constructor, and shall not be an aggregate.~~

§ 5 References

[CWG2525] Jim X. 2021-09-25. Incorrect definition of implicit conversion sequence. https://wg21.link/cwg2525

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++. https://wg21.link/n4971

---

1. All citations to the Standard are to working draft N4971 unless otherwise specified.↩